Lab: Connecting a PHP Application to a MySQL Container
To Begin with the Lab

Summary

In this lab, we will learn how to establish communication between two Docker containers: one running a PHP application with Apache and another running a MySQL database server.

Our PHP application will connect to the MySQL database, retrieve course information from a table, and display the records on a web page.

We will also learn how to install the MySQL extension required by PHP, configure the database connection, rebuild the Docker image, and verify that changes made in MySQL are reflected in the PHP application.

Understanding Container-to-Container Communication

When we run multiple containers using Docker, they can communicate with each other over a Docker network.

In this lab, we have two containers:

1. **PHP Container:** Runs the PHP application using Apache.
2. **MySQL Container:** Runs the MySQL database server and stores course information.

The PHP application needs the MySQL connection details to retrieve data from the database.

These include:

- Database host IP address or hostname.
- MySQL username.
- MySQL password.
- Database name.
- Required PHP MySQL extension.

Example:

Suppose we have a PHP application that displays course information.

The course details are stored in a MySQL database named app_db, inside a table named courses.

The PHP application connects to MySQL, executes a SELECT query, retrieves the course ID, course name, and rating, and displays the results in an HTML table.

Lab Steps

Step 1: Prepare the MySQL Database

• Open MySQL Workbench on Windows.

• Connect to your MySQL Docker container.

• Create a database:

Note: These are sample records for practicing container communication.

Step 2: Identify the MySQL Container IP Address

• Open your Ubuntu WSL terminal.

• List the running containers:

```
docker ps
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                                  NAMES
bef7aa400825   mysql     "docker-entrypoint.s…"   2 minutes ago   Up 49 seconds   127.0.0.1:3306->3306/tcp, [::1]:3306->3306/tcp   mysql-instance
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Inspect the MySQL container:

```
docker inspect mysql-instance
```

• Find the IPAddress field in the NetworkSettings section.

Example:

"IPAddress": "172.17.0.3"

This is the container's IP address on its Docker network.

Understanding:

In the demonstration, the PHP application uses this IP address as the database host to connect directly to the MySQL container.

Important: Container IP addresses can change when containers are recreated. In practical applications, it is better to create a custom Docker network and connect using the MySQL container name instead of relying on a fixed IP address.

Step 3: Configure the PHP Application

• Open your PHP project folder.

• Open the index.php file.

• Configure the database connection using PHP's MySQLi extension.

Index.php

```php
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="description" content="Displaying list of courses">
    <link href="./lib/bootstrap/dist/css/bootstrap.min.css" rel="stylesheet">
    <title>List of Employees</title>
</head>
<body>
<?php
    $host_name = '172.17.0.2';
    $user = 'root';
    $password = 'sqlset1010';
    $database='appdb';
    $connection = new mysqli($host_name, $user, $password,$database);

    if($connection) {
     $statement = " SELECT Employee ID,Employee Name,Phone FROM Employee Information ";
     $records = $connection->query($statement);
     $connection->close();
```

Explanation:

PHP Code	Description
new mysqli()	Establishes a connection to MySQL.
$host	MySQL container IP address in this example.
$username	MySQL username.
$password	MySQL password.
$database	Database name.
$conn->query()	Executes the SQL query.
$result->fetch_assoc()	Retrieves records as associative arrays.
$conn->close()	Closes the database connection.

Step 4: Modify the Dockerfile

The default PHP Apache image does not necessarily include the MySQLi extension.

We need to install it so that PHP can communicate with MySQL.

• Open your project's Dockerfile.

• Add the RUN instruction:

```
FROM php:8.2-apache
RUN docker-php-ext-install mysqli
COPY . /var/www/html
EXPOSE 80
CMD ["apache2-foreground"]
```

Understanding the RUN instruction:

The RUN instruction executes commands while Docker builds an image.

Here:

```
RUN docker-php-ext-install mysqli
```

installs the MySQLi extension required by our PHP application.

Important: The RUN instruction executes during image building, not when the container starts.

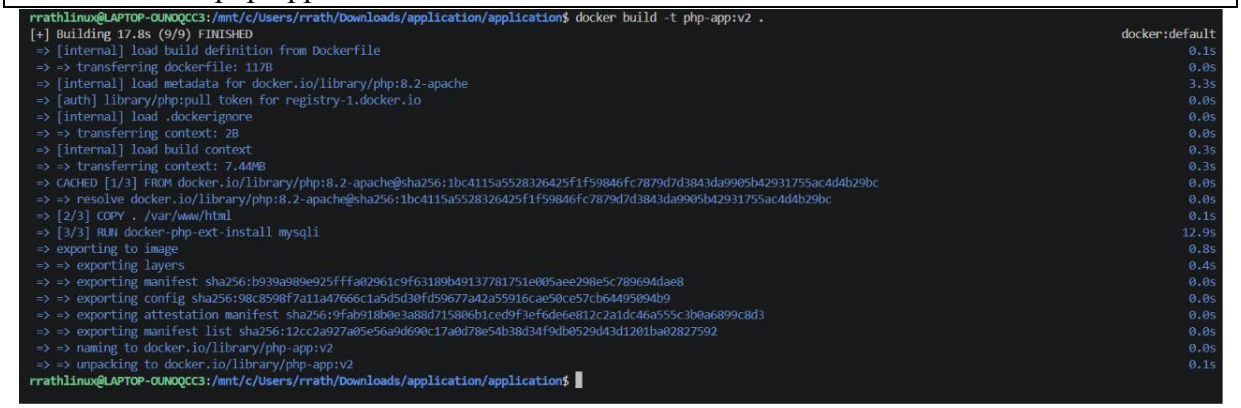
Step 5: Build a New PHP Docker Image

Since we modified the PHP code and Dockerfile, we need to create a new Docker image.

• Navigate to the project directory containing the Dockerfile.

• Build the new image:

```
docker build -t php-app:v2 .
```



Explanation:

Command	Description
docker build	Builds a Docker image.
-t	Assigns a name and tag.
php-app	Image name.
v2	Image version.
.	Uses the current directory as the build context.

Step 6: Remove the Old PHP Container

• Check existing containers:

```
docker ps -a
```

• Stop the old PHP container:

```
docker stop php-instance
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker stop mysql-instance
mysql-instance
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Remove the old PHP container:

```
docker rm php-instance
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker rm mysql-instance
mysql-instance
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Replace php-container with the actual name of your old PHP container.

Note: Do not remove the MySQL container because we still need it to store and provide the database records.

Step 7: Run the New PHP Container

• Create a new container using the updated image:

```
docker run -d --name php-innstance -p 8080:80 php-app:v2
```

Explanation:

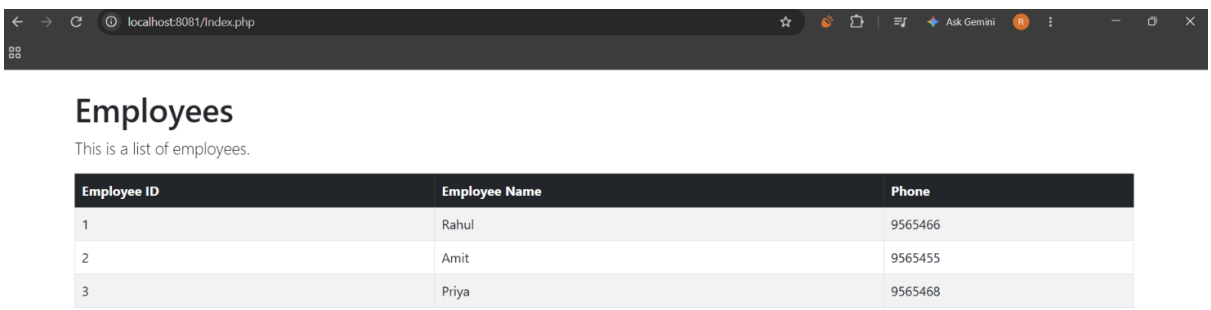
- -d: Runs the container in detached mode.
- --name: Assigns a name to the container.
- -p 8080:80: Maps host port 8080 to container port 80.
- php-app:v2: Uses the newly created PHP image.

Step 8: Test the PHP Application

• Open your browser.

• Navigate to:

http://localhost:8080



You should see the course information retrieved from the MySQL database.

The records displayed on the web page are coming from the MySQL database running inside another container.

Step 9: Verify Database Communication

Now we will verify whether the PHP application is actually fetching live information from MySQL.

• Open MySQL Workbench.

• Connect to the MySQL Docker container.

• Execute the following query: